Adaptive and Concurrent Garbage Collection for Virtual Machines

Md Enamul Haque, SM Zobaed, Razin Farhan Hussain
Aminul Islam, and Christian Traylor

School of Computing and Informatics
University of Louisiana, Lafayette, LA 70504, USA

January 10, 2020
Grand Wailea, Maui, HI

Overview

1. Introduction.
2. Motivation.
3. Problem Statement.
4. Proposed Approach.
5. Experimental Evaluations.

Introduction

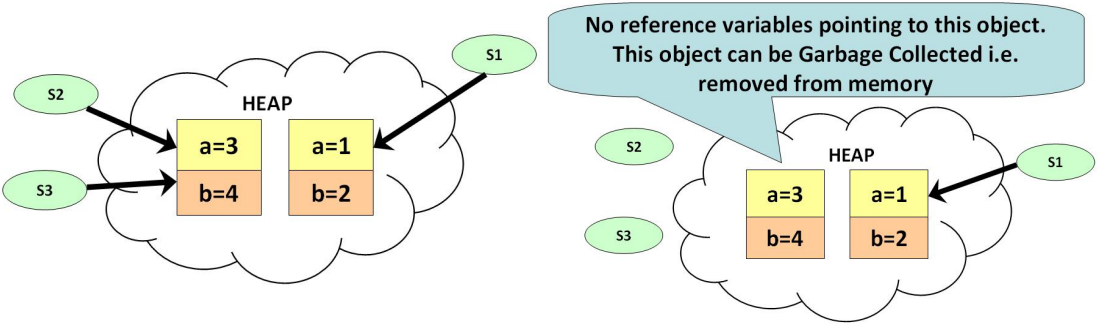


Figure: Toy example of a garbage collection scenario.

Introduction

Motivation

- Various GC techniques are widely adopted.
- Program specific GC works better.
- Selecting the best GC algorithm for every program is a challenging task.
  - Exhaustive applications profiling.
  - Learning program specific historical GC usage.

Problem Statement

Considering *execution time* $\mathcal{E}(p_j, c_i)$ and *garbage collection time* $\mathcal{G}(p_j, c_i)$ as functions of program and garbage collection, we present our optimization function in Equation 1.

$$\arg\min_{c_i} \left[\mathcal{E}(p_j, c_i) + \mathcal{G}(p_j, c_i)\right] \tag{1}$$

Execution time is dependent on processor configuration.

$$\arg\min_{c_i} \mathcal{G}(p_j, c_i) \tag{2}$$

Proposed Approach

ACGC Process

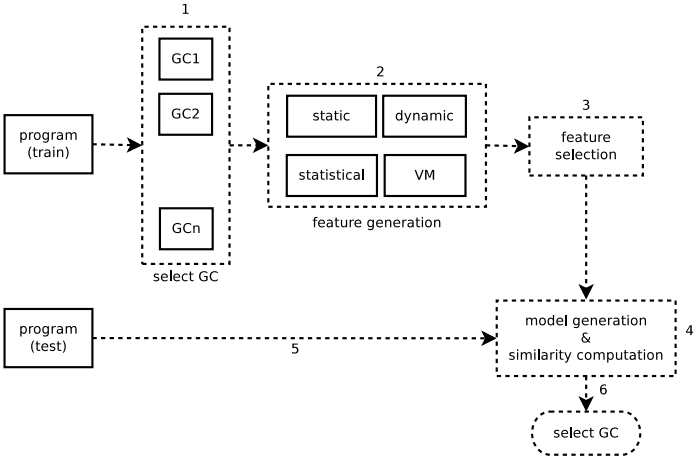


Figure: **Process diagram of adaptive and concurrent GC (ACGC)**

GC Algorithms

- Serial GC: Single threaded, simple heap
- Parallel GC: Parallel threaded, minor collection parallel, Good when sufficient main memory and large number of cores
- G1 GC: Server-side, multi-processor
- Concurrent Mark Sweep (CMS): CMS collector is designed for applications that prefer shorter garbage collection pauses
- Par New GC: Young generation is normally small in size, the collection should be very quick

Feature Generation

- Static metrics (*ckjm* suite):
  1. depth of inheritance tree (DIT),
  2. weighted methods per class (WMC),
  3. number of children (NOC),
  4. coupling between object classes (CBO),
  5. response for a class (RFC),
  6. lack of cohesion methods (LCOM),
  7. afferent couplings (Ca),
  8. number of public methods (NPM)
- Dynamic metrics on reference VM:
  1. number of allocated objects,
  2. average of allocated object space,
  3. execution time for an arrangement
- VM metrics: heap size
- Statistical metrics: Entropy and Complexity

Non-negative Matrix Factorization (NMF)

$$\arg \min_{X^l, Y^l > 0} \frac{1}{2} ||X - X^l Y^l||_F^2 \tag{3}$$

where feature matrix X is transformed into a l dimensional feature space with two components X^l and Y^l such that the approximation in Equation 4 holds.

$$X \approx X^l Y^l \tag{4}$$

Algorithm1: ACGC+Cosine

Algorithm 1: ACGC using non-negative matrix factorization as feature generation with cosine similarity.

Input: Training data $\mathcal{T} = \{\mathcal{X}^{train}, \mathcal{Y}\}$,
Test data $\mathcal{D} = \mathcal{X}^{test}$
Set: Latent factors, l, $\mathcal{Y}^{test} = \emptyset$
Output: Optimal GC set $\mathcal{Y}^{test}$
Training:
1. Factorize $\mathcal{X}^{train}$ into l dimensional latent space to get X^l and Y^l components using Equation 3.

Test:
repeat
1. Randomly select a test program feature x^{test}.
2. Compute similarity score σ_{test} from x^{test} to all the features in X^l using cosine similarity as below

$$\sigma_{i,j}^{test} = \frac{\sum_{k=1}^{l} x_{ik}^{test} X_{jk}^{l}}{\sqrt{\sum_{k=1}^{l} x_{ik}^{test\,2}}\sqrt{\sum_{k=1}^{l} X_{jk}^{l\;2}}}$$

3. Calculate majority vote to select Y^{best} using σ_{test} for x^{test}.
4. Set $\mathcal{Y}^{test} = \mathcal{Y}^{test} \cup Y^{best}$.

until $\mathcal{D} = \emptyset$;
return $\mathcal{Y}^{test}$

Algorithm1: ACGC+Learner

Algorithm 2: ACGC using non-negative matrix factorization as feature generation with learning algorithms.

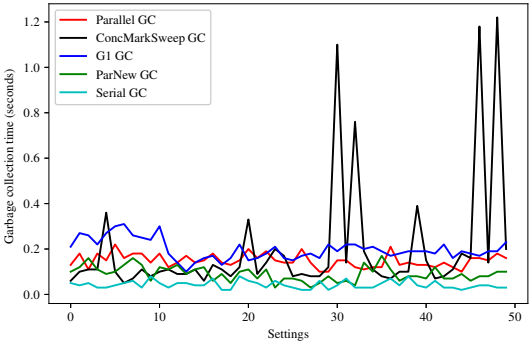
Input: Training data $\mathcal{T} = \{\mathcal{X}^{train}, \mathcal{Y}\}$,
Test data $\mathcal{D} = \mathcal{X}^{test}$
Set: Latent factors, l, $\mathcal{Y}^{test} = \emptyset$
Output: Optimal GC set $\mathcal{Y}^{test}$
Training:
1. Factorize $\mathcal{X}^{train}$ into l dimensional latent space to get X^l and Y^l components using Equation 3.
Test:
repeat
  1. Randomly select a test program feature x^{test}.
  2. Compute Y^{best} using learning function.
  3. Set $\mathcal{Y}^{test} = \mathcal{Y}^{test} \cup Y^{best}$.
until $\mathcal{D} = \emptyset$;
return $\mathcal{Y}^{test}$

Experimental Evaluations

Program Benchmarks

Suite	Benchmark	Description
JOlden	bh	Mathematical computation
	bisort	Sorting
	em3d	Mathematical computation
	health	Process simulation
	mst	Minimum spanning tree
	perimeter	Image processing
	power	Process simulation
	treeadd	Tree traversal
	tsp	Graph optimization
	voronoi	Image processing
SPECJvm2008	compiler	Front end Java compiler
	compress	Data compression
	crypto	Cryptographic algorithms
	derby	Database logic and lock computation
	MPEGaudio	Mp3 decoder
	scimark	Floating point computation
	serial	Serialization and de-serialization of objects
	startup	Multi-threaded optimization
	sunflow	Graphics visualization
	xml	XML transformation
Dacapo	avrora	AVR microcontrollers
	tomcat	Web server
	batik	Scalable Vector Graphics (SVG) images
	eclipse	JDT performance tests
	xalan	XML to HTML converter
	fop	PDF generator

Dataset Properties w.r.t Heap Size



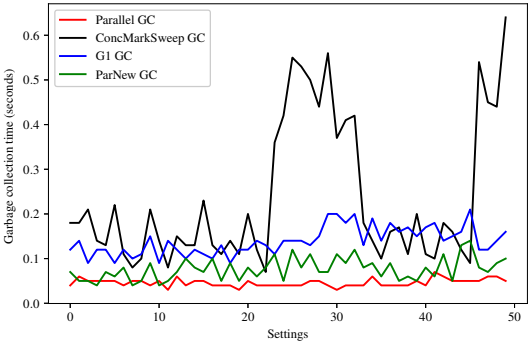


Figure: **Garbage collection time behavior for "scimark" and "compress" in SPEC2008 benchmark.**

Experimental Evaluation (Algorithm#1)

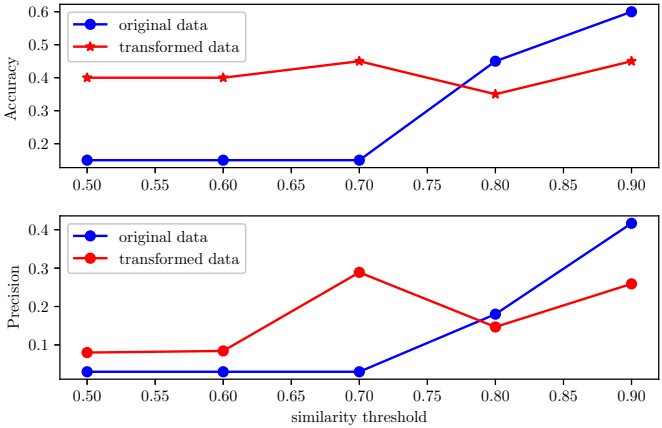


Figure: **Accuracy and precision score for original and transformed data with respect to varying threshold similarity (`Jolden`).**

Experimental Evaluation (Algorithm#2)

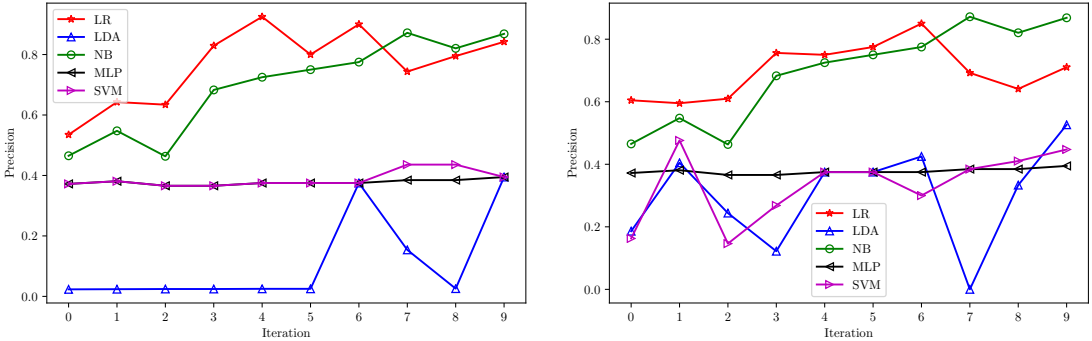


Figure: **Precision values of different classification algorithms for `Dacapo` benchmark for (a) Original (b) Transformed data**

Experimental Evaluation (Algorithm#2 contd.)

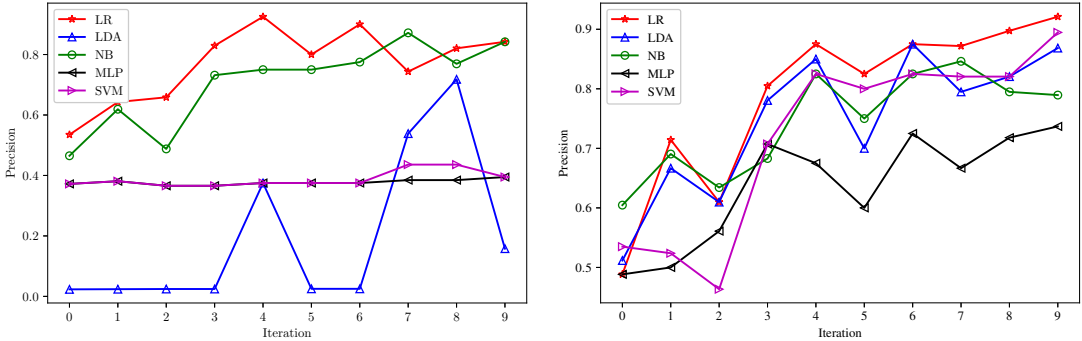


Figure: **Precision values of different classification algorithms for `Jolden` benchmark for (a) Original (b) Transformed data**

Comparative Results

Table: **Precision and accuracy scores of two approaches for two versions of learning for `Jolden` and `Dacapo` data. (max and sd values are reported)**

Benchmark	Algorithms	**Original**		**Transformed**	
		Accuracy	Precision	Accuracy	Precision
Jolden	ACGC+cosine (Alg1)	0.59±0.26	0.40±0.18	0.45±0.20	0.28±0.08
	ACGC+learning (Alg2)	0.84±0.28	0.83±0.22	0.95±0.33	0.94±0.32
Dacapo	ACGC+cosine (Alg1)	0.48±0.12	0.43±0.15	0.56±0.18	0.52±0.21
	ACGC+learning (Alg2)	0.86±0.21	0.76±0.23	0.94±0.27	0.89±0.28

Conclusions

Optimum GC selection can be done without exhaustive profiling

- Original feature + cosine → Bad
- Factored feature + cosine → Good & Bad
- Factored feature + Learner → Great!

Questions?

https://github.com/enamul-haque/ACGC